

COMMERCIAL DIGITAL STUDY PRODUCT

Practice File Lab

Programming drills, review sheets, debugging logs, and data-structure exercises.

Prepared by DevShelf Academy

DevShelf Academy owns this compiled commercial study product, including the product layout, original explanations, exercises, worksheets, project prompts, checklists, and delivery package.

Referenced open educational works remain owned by their respective authors and are credited in the license section. This product does not claim ownership of those works and is not a resale of any third-party book as-is.

Product price: EUR 245

TASK	BUYER ACTION
1	Use this PDF as a study workbook, not a passive reading file.
2	Every lesson page connects an open educational source theme with a new DevShelf exercise.
3	Code along, create notes, and keep the final review pages with your project files.
4	Follow the attribution notes if you redistribute any adapted share-alike material.



What This Pack Contains

This file combines source-informed programming topics with DevShelf Academy's own learning flow. The public sources provide topic framing and licensing transparency; the actual sequence, explanations, worksheets, review prompts, and commercial product presentation are original to DevShelf Academy.

Source-informed tracks

PYTHON - informed by Dive Into Python 3

DATA - informed by Open Data Structures

GRAPH - informed by Introduction to Graph Theory

CS - informed by Structure and Interpretation of Computer Programs

Ownership and licensing

DevShelf Academy owns this compiled commercial study product, including the product layout, original explanations, exercises, worksheets, project prompts, checklists, and delivery package.

Referenced open educational works remain owned by their respective authors and are credited in the license section. This product does not claim ownership of those works and is not a resale of any third-party book as-is.

Product price: EUR 245

LESSON 001 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 002 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 003 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 004 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 005 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 006 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 007 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 008 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 009 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 010 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 011 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 012 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 013 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 014 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 015 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 016 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 017 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 018 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 019 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 020 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 021 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 022 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 023 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 024 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 025 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 026 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 027 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 028 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 029 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 030 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 031 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 032 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 033 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 034 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 035 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 036 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 037 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 038 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 039 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 040 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 041 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 042 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 043 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 044 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 045 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 046 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 047 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 048 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 049 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 050 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 051 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 052 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 053 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 054 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 055 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 056 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 057 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 058 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 059 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 060 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 061 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):
    items = clean_input(raw_text)
    checked = validate(items)
    return format_result(checked)
```

LESSON 062 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 063 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 064 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 065 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 066 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 067 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 068 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 069 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 070 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 071 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 072 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 073 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 074 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 075 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 076 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 077 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 078 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 079 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 080 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 081 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 082 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 083 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 084 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 085 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 086 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 087 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 088 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 089 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 090 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 091 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 092 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 093 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 094 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 095 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 096 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 097 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 098 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 099 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 100 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 101 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 102 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 103 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 104 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 105 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):
    items = clean_input(raw_text)
    checked = validate(items)
    return format_result(checked)
```

LESSON 106 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 107 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 108 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 109 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 110 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 111 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 112 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 113 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):
    items = clean_input(raw_text)
    checked = validate(items)
    return format_result(checked)
```

LESSON 114 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 115 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 116 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 117 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 118 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 119 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 120 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 121 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 122 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 123 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 124 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 125 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 126 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 127 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 128 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 129 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 130 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 131 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 132 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 133 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 134 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 135 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 136 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 137 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 138 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 139 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 140 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 141 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 142 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 143 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 144 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 145 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 146 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 147 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 148 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 149 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 150 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 151 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 152 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 153 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 154 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 155 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 156 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 157 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 158 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 159 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 160 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 161 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 162 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 163 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 164 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 165 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 166 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 167 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 168 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 169 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 170 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 171 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 172 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 173 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 174 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 175 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 176 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 177 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 178 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 179 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 180 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 181 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 182 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 183 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 184 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 185 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 186 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 187 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 188 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 189 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 190 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 191 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 192 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 193 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 194 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 195 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 196 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 197 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 198 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 199 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 200 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 201 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 202 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 203 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 204 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 205 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 206 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 207 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 208 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 209 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 210 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 211 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 212 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 213 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 214 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 215 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 216 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 217 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 218 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 219 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 220 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 221 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 222 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 223 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 224 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 225 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):
    items = clean_input(raw_text)
    checked = validate(items)
    return format_result(checked)
```

LESSON 226 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 227 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 228 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 229 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 230 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 231 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 232 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 233 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 234 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 235 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 236 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 237 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a file naming helper.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a file naming helper that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 238 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a priority list for review topics.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a priority list for review topics that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 239 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a connected-components check for isolated notes.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a connected-components check for isolated notes that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 240 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a higher-order helper for repeated validation rules.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a higher-order helper for repeated validation rules that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 241 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a command-line note cleaner.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a command-line note cleaner that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):
    items = clean_input(raw_text)
    checked = validate(items)
    return format_result(checked)
```

LESSON 242 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a dictionary-backed glossary.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a dictionary-backed glossary that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 243 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a route between concepts using edges.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a route between concepts using edges that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 244 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a stateful counter with a clear mutation boundary.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a stateful counter with a clear mutation boundary that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 245 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a study progress tracker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a study progress tracker that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 246 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of correctness, time complexity, space complexity, asymptotic notation, and practical tradeoffs. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a task queue for pending study sessions.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a task queue for pending study sessions that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 247 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a dependency graph for project files.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a dependency graph for project files that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 248 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a stream-like process that handles one record at a time.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a stream-like process that handles one record at a time that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 249 | PYTHON

Python setup and first program

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about python setup and first program, with a practical scenario: a quiz checker.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, python setup and first program should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to linked lists. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of python setup and first program without looking at the source.
2	Build a 20-line example for a quiz checker that demonstrates turn environment setup into a repeatable launch checklist.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use python setup and first program, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checked)
```

LESSON 250 | DATA

Array-backed lists

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of interfaces for stacks, queues, lists, unordered sets, and sorted sets. DevShelf Academy converts that public learning direction into an original workbook page about array-backed lists, with a practical scenario: a graph of related programming concepts.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, array-backed lists should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to trees as graphs. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of array-backed lists without looking at the source.
2	Build a 20-line example for a graph of related programming concepts that demonstrates understand indexed access, resizing, and shifting costs.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use array-backed lists, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```

LESSON 251 | GRAPH

Connectedness

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about connectedness, with a practical scenario: a map of prerequisites between study topics.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, connectedness should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to higher-order thinking. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of connectedness without looking at the source.
2	Build a 20-line example for a map of prerequisites between study topics that demonstrates ask whether every important item can reach another.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use connectedness, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 252 | CS

Recursion

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about recursion, with a practical scenario: a recursive folder walk described before it is coded.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, recursion should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to unit testing. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of recursion without looking at the source.
2	Build a 20-line example for a recursive folder walk described before it is coded that demonstrates describe a problem in terms of a smaller problem.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use recursion, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

LESSON 253 | PYTHON

Files and paths

SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/

Source-informed concept

Source lens: Dive Into Python 3 is used here for the topic area of Python installation, first programs, native data types, comprehensions, strings, files, unit testing, refactoring, packaging, and HTTP services. DevShelf Academy converts that public learning direction into an original workbook page about files and paths, with a practical scenario: a small data parser.

DevShelf explanation

Python turns ideas into short executable experiments, so every lesson should end with a running script or a tested function. In this lesson, files and paths should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: ignoring failed input and letting the program crash silently.

Applied connection

Connect this lesson to sorting and comparison. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of files and paths without looking at the source.
2	Build a 20-line example for a small data parser that demonstrates separate path selection, file reading, parsing, and output.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use files and paths, and when should they avoid it?

Example workflow shape

```
def run_tool(raw_text):  
    items = clean_input(raw_text)  
    checked = validate(items)  
    return format_result(checkered)
```

LESSON 254 | DATA

Heaps and priority queues

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/

Source-informed concept

Source lens: Open Data Structures is used here for the topic area of array-backed lists, linked lists, hash tables, binary trees, heaps, sorting, and graph representations. DevShelf Academy converts that public learning direction into an original workbook page about heaps and priority queues, with a practical scenario: a searchable archive of notes.

DevShelf explanation

Data structures are choices about behavior, cost, and clarity, not just names to memorize. In this lesson, heaps and priority queues should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: measuring only one input size and trusting the result.

Applied connection

Connect this lesson to algorithmic graph checks. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of heaps and priority queues without looking at the source.
2	Build a 20-line example for a searchable archive of notes that demonstrates retrieve the most important item efficiently.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use heaps and priority queues, and when should they avoid it?

Example workflow shape

```
structure = choose_for(main_operation)
for item in incoming_items:
    structure.add(item)
answer = structure.query(target)
```


LESSON 255 | GRAPH

Proof habit

SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Source-informed concept

Source lens: Introduction to Graph Theory is used here for the topic area of vertices, edges, walks, paths, cycles, connectedness, graph representations, and proof habits. DevShelf Academy converts that public learning direction into an original workbook page about proof habit, with a practical scenario: a cycle check for tasks that depend on each other.

DevShelf explanation

Graph thinking turns relationships into data that can be searched, validated, and explained. In this lesson, proof habit should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: forgetting to record visited nodes.

Applied connection

Connect this lesson to abstraction layers. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of proof habit without looking at the source.
2	Build a 20-line example for a cycle check for tasks that depend on each other that demonstrates state assumptions before trying to prove a property.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use proof habit, and when should they avoid it?

Example workflow shape

```
visited = set()
frontier = [start]
while frontier:
    node = frontier.pop()
```

LESSON 256 | CS

Evaluation models

SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf

Source-informed concept

Source lens: Structure and Interpretation of Computer Programs is used here for the topic area of building programs by controlling complexity through levels of abstraction. DevShelf Academy converts that public learning direction into an original workbook page about evaluation models, with a practical scenario: a calculator broken into expression, operation, and output layers.

DevShelf explanation

Computer science fundamentals help reduce complexity by naming the right abstraction level. In this lesson, evaluation models should be studied through a working artifact, not through passive reading. Start with the scenario, define the input and output, then decide which part of the program needs the concept. A useful learner's answer should include the concept's purpose, the cost it introduces, and the failure mode it helps prevent. Watch for this common mistake: using recursion without identifying the base case.

Applied connection

Connect this lesson to native data types. For example, build the first version with the simplest implementation, then add the adjacent idea only when it removes duplication, clarifies ownership, improves lookup, or makes testing easier. This is how the public-source topic becomes a custom DevShelf practice path.

Practice checklist

TASK	BUYER ACTION
1	Write a five-sentence explanation of evaluation models without looking at the source.
2	Build a 20-line example for a calculator broken into expression, operation, and output layers that demonstrates trace how an expression becomes a result.
3	Add one invalid input or edge case and describe the expected behavior.
4	Refactor the example once for naming and once for simpler control flow.
5	Write a final note: when should a learner use evaluation models, and when should they avoid it?

Example workflow shape

```
define representation
define constructor
define selector
hide the internal details
```

Final Project Plan

Choose one product-sized project and connect at least three tracks from this PDF. For example: a Python command-line study tracker using dictionaries, file output, basic tests, and a graph-like relationship map between topics.

TASK	BUYER ACTION
1	Define the user action and visible output.
2	Select the data structure before writing code.
3	Write one test for normal input and one test for invalid input.
4	Document one refactor and explain what became simpler.
5	Write a final attribution note if your project includes adapted source material.

Attribution and License Notes

The following sources were used for topic selection and educational framing. The DevShelf Academy product is a newly compiled study guide with original lesson text, exercises, worksheets, project prompts, and layout.

SOURCE	Open Data Structures
LICENSE	Creative Commons Attribution 2.5
URL	https://opendatastructures.org/
SOURCE	Dive Into Python 3
LICENSE	Creative Commons Attribution-ShareAlike
URL	https://diveintopython3.net/
SOURCE	Comprehensive Rust
LICENSE	CC BY 4.0
URL	https://google.github.io/comprehensive-rust/
SOURCE	Structure and Interpretation of Computer Programs
LICENSE	CC BY-SA 4.0
URL	https://web.mit.edu/6.001/6.037/sicp.pdf
SOURCE	Introduction to Graph Theory
LICENSE	CC0 1.0
URL	https://www.cip.ifi.lmu.de/~grinberg/t/22s/graphs.pdf

Commercial use note

Only sources with commercial-friendly licenses were selected for this product family. NonCommercial sources were intentionally excluded from the product content.

Customer license

Customers may use this purchased DevShelf Academy PDF for personal study and internal learning. The PDF itself, its commercial packaging, original lesson sequence, and DevShelf worksheets may not be resold as a competing product.